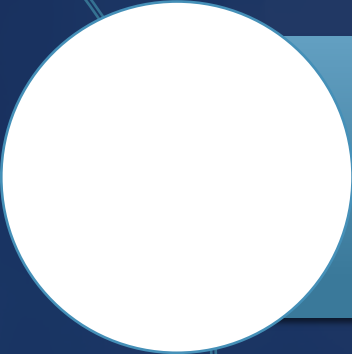




SISTEMAS OPERACIONAIS

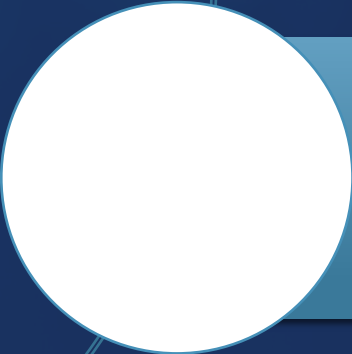
AULA 8 THREADS

CONTEXTUALIZANDO



Aula passada:

Escalonamento de Processos



Aula de hoje:

Threads

MODELO DE PROCESSO

- 1) Utilizado para agrupar recursos
- 2) Um espaço de endereço (de 0 até algum endereço máximo do processo) e uma única linha execução → **Thread**
- 3) Agrupamento de recursos (espaços de endereço com texto e dados do programa, arquivos abertos, processos filhos, tratadores de sinais, alarmes pendentes, etc.)

MODELO DE THREADS

- 1) Um espaço de endereço e múltiplas linhas de controle
- 2) Conjunto de threads compõe as linhas de execuções de um processo
- 3) Threads compartilham um mesmo espaço de endereço, sendo menos independentes que processos
- 4) Possuem recursos particulares: PC (Process Counter), registradores, pilha ...

PROCESSOS & THREADS

Um **processo** e uma **thread** são conceitos fundamentais de sistemas operacionais e programação concorrente.

A forma mais simples de entender é:

Processo = um programa em execução, com seu próprio espaço de memória e recursos.

Thread = uma linha de execução dentro de um processo.

PROCESSOS

Um processo possui:

- espaço próprio de memória;
- variáveis próprias;
- recursos do sistema;
- arquivos abertos;
- identificador (PID);
- contexto de execução próprio.

Quando você abre:

- navegador,
- VSCode,
- Spotify,

Cada um normalmente roda em um processo separado.

Se um processo falha, os outros processos continuam funcionando normalmente.

CARACTERÍSTICAS IMPORTANTES DE PROCESSO

- 1) Mais isolado e seguro.
- 2) Comunicação entre processos é mais complexa.
- 3) Criar processos custa mais recursos do sistema.

THREAD

- 1) Uma thread é uma unidade de execução dentro de um processo
- 2) Um mesmo processo pode possuir várias threads compartilhando:
 - **memória**
 - **variáveis globais**
 - **recursos do processo**
 - **arquivos abertos**

THREAD

Mas cada thread possui:

- contador de programa
- pilha (stack)
- registradores
- estado de execução próprios.

THREAD → VANTAGENS

- 1) Em muitas aplicações há múltiplas atividades ao mesmo tempo
- 2) Podemos decompô-las em atividades paralelas
- 3) Algumas tarefas precisam do compartilhamento do espaço de endereçamento
- 4) CPU-bound e I/O-bound podem se sobrepor, acelerando a aplicação

THREAD → VANTAGENS

- 1) São mais rápidas de criar e destruir que processos
- 2) Em algumas ocasiões são até 100 vezes mais rápidas
- 3) Úteis em sistemas com múltiplas CPUs (arquitetura multi-core: **paralelismo real**)

ANALOGIA PRÁTICA

Imagine um restaurante

Processo → Restaurante

Um restaurante possui:

- cozinha,
- estoque,
- mesas,
- energia,
- recursos próprios.

ANALOGIA PRÁTICA

Threads → Funcionários

Os cozinheiros e garçons:

- trabalham ao mesmo tempo;
- compartilham os recursos do restaurante;
- executam tarefas diferentes simultaneamente.

Se um funcionário erra, o restaurante pode continuar funcionando. Mas se o restaurante pega fogo, processo encerra, todos os funcionários param.

PROCESSO X THREAD

Comparação direta

Aspecto	Processo	Thread
Memória	Separada	Compartilhada
Criação	Mais pesada	Mais leve
Comunicação	Mais difícil	Mais fácil
Isolamento	Alto	Baixo
Troca de contexto	Mais lenta	Mais rápida
Falha	Afeta só o processo	Pode derrubar o processo inteiro

EXEMPLO REAL NO NAVEGADOR

O Google Chrome usa muitos processos e threads

- Cada aba pode ser um processo separado
- Renderização gráfica pode usar threads
- Reprodução de vídeo pode usar outras threads
- Rede pode usar threads independentes

Isso melhora: desempenho, paralelismo, estabilidade

EM SISTEMAS OPERACIONAIS

○ Sistema Operacional agenda:

- PROCESSOS
- THREADS

A CPU executa

- THREADS

Ou seja: **o que roda realmente na CPU são as THREADS.**

○ processo funciona mais como um container de recursos

EXEMPLO EM PROGRAMAÇÃO

Processo único com múltiplas threads

Um videogame pode ter:

- uma thread para áudio
- outra para física
- outra para renderização
- outra para rede

Tudo dentro do mesmo processo

RELAÇÃO COM CPUS MODERNAS

Em CPUs **multicore**:

Diferentes threads podem executar simultaneamente em núcleos diferentes.

Exemplo:

CPU de 8 núcleos → pode executar várias threads ao mesmo tempo

RESUMO FINAL

Processo:

- programa em execução
- isolado
- possui recursos próprios

Thread:

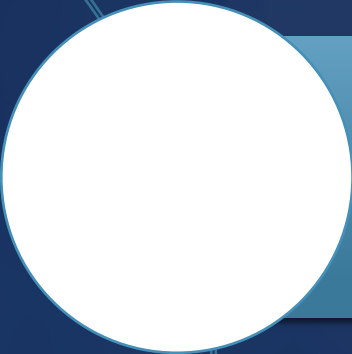
- fluxo de execução dentro do processo
- compartilha memória
- mais leve e rápida

RESUMO FINAL

PROCESSOS são independentes

**THREADS cooperam dentro
do mesmo processo.**

CONCLUINDO



Vimos nesta aula:

THREADS



Na próxima aula:

Comunicação Interprocessos - IPC



PROFESSOR
PARISOTTO

www.parisotto.com.br